

用 Neocarta 从 GCP 建一个语义层 Build a Semantic Layer from GCP with Neocarta

本篇范围说明：这是 Neo4j 开发者博客 2026-06-11 的一篇操作教程（作者 Alex Gilmore，Neo4j 高级 AI 解决方案架构师，标称 14 分钟阅读）。下面「全文翻译」一节覆盖它的架构说明、图数据模型、数据来源、以及三个问答示例；命令行安装步骤、环境变量清单、MCP 配置 JSON 这类可以照抄的操作细节不逐段翻译，保留原文形式即可。

核心内容

Q: 这篇文章在做什么？

A: 它演示怎么用 Neocarta 这个 Python 库，把 GCP 上 BigQuery 的表结构信息和 Knowledge Catalog 里的业务术语表，一起导进 Neo4j 变成一张「语义层图」，然后接一个 MCP 服务让智能体用这张图去回答关于数据的问题。Neocarta 是 Neo4j Labs 的项目，还在快速开发中。

Q: 它说的「语义层」和一般说的语义层有什么不同？

A: 作者给的定义里，**关键属性是「众多互不相同的数据资产之间的关系」**。他举的问句是：一个列名对应到哪个业务术语？还有别的列共享这个含义吗？这些列怎么组合起来能得出新的洞察？他的主张是，一个能访问「图支撑的语义层」的智能体，在一片数据海洋里不难把点连起来、知道该去哪里找答案。

Q: 这张图长什么样？

A: 两个主要部分，分层叠在一起。这张表回答一个问题：每一层装什么、这一层的节点类型来自哪个数据来源。

层	节点类型	数据来源	这一层解决什么
数据库元数据层	Database、Schema、Table、Column	BigQuery（Database 对应 GCP 项目，Schema 对应 BigQuery 数据集）	显式地把库、模式、表、列的层级关系画出来
术语表	Glossary、Category、BusinessTerm	Knowledge Catalog（原 Dataplex Universal Catalog）	提供业务术语，让检索能按业务说法找到数据资产
查询日志补充	Query、CTE，以及额外的列间关系	BigQuery 查询日志	补上外键定义里没有、但能从 SQL 的 JOIN 条件推断出来的关系

读法：拿第三行走一遍。前两层里表和列之间的关系来自外键定义，而现实中很多表之间的关联从来没有声明成外键——它只存在于人们实际写的 SQL 里。所以第三层去解析查询日志，把 JOIN 条件里出现过的列对当成一条关系加进图里。

两层之间怎么连：BusinessTerm 节点通过 TAGGED_WITH 关系连到 Table 和 Column 节点上。所以数据资产可以经由它和业务术语的连接被找到。

Q：智能体怎么用这张图？

A：Neocarta 的 MCP 服务会按图里实际存在的索引和实体去注册工具。因为这张图有向量索引、全文索引和 BusinessTerm 节点，它注册了五个工具：列出模式、按模式列出表、按「表加业务术语」混合检索取上下文、按「列加业务术语」混合检索取上下文、以及取完整元数据模式（只用于开发调试）。

其中「混合检索加业务术语」那两个工具的检索路径值得单独看，它做三次检索来确定锚点节点池：主实体的向量检索、BusinessTerm 的全文检索、主实体的全文检索。

全文检索那一路是两跳的：先找到 BusinessTerm 节点，再顺着连接走到相连的实体节点（表或列），然后在这些实体节点上再跑一次全文检索，算归一化平均分。这个结果再和向量检索的结果合并、按分排序、取前 k 个。

Q：文章给的三个问答示例说明了什么？

A：三个问题的难度是递增的，而它们说明的是**语义层让智能体在保持上下文很轻的前提下回答越来越复杂的问题：**

第一个「哪条产品线对总年度经常性收入贡献最大」，用到两张表；第二个「最近一个考核周期里各部门的绩效评级分布」，需要三张表；第三个「客户健康分和他们的支持体验之间有相关性吗」，需要连接两张表并推断情况。

第三个问题的回答里有一个细节值得注意：智能体不只给了相关性，还指出「客服满意度低于 4.0 或解决时长超过 30 小时的客户值得客户成功经理主动接触」——它给的是一个可执行的阈值，而不只是一个统计描述。

背景补充 / 点评

这篇文章的定位要说清楚：它是一篇产品教程，不是一篇方法论文章。 Neocarta 是 Neo4j Labs 的项目，文章作者是 Neo4j 的解决方案架构师，全文的骨架是「装依赖、跑命令、看结果」。所以它的价值不在于论证「为什么该这样建语义层」，而在于**它把一个能跑的语义层的最小数据模型摊开了**——那个数据模型比论证更值得看。

先说这个数据模型里我认为最实在的一处：解析查询日志来补关系。

外键定义是设计时写下的意图，而 JOIN 条件是运行时实际发生的连接。**两者不一致时，后者才是真实的数据关系。** 很多生产库里根本不建外键（性能考虑、或者跨库没法建），于是「哪两张表实际上是关联的」这个知

识只存在于人们写过的 SQL 里。Neocarta 把这部分挖出来变成图里的边，这一步很聪明。

但这一步也有一个它没提的风险：从 JOIN 条件推断出来的关系不区分「这是一个正确的连接」和「这是一个有人写错但跑通了的连接」。一个用错列做的 JOIN 照样会出现在日志里、照样会被推断成一条关系。文章没有提任何过滤或置信度机制，而这类关系一旦进了语义层，智能体就会照着它去生成查询。

再说术语表那一层，它解决的是一个真问题但方式偏弱。BusinessTerm 通过 TAGGED_WITH 连到表和列，这让「按业务说法找数据」成为可能。但 TAGGED_WITH 是一种无向、无类型的标注关系——它说不出「这一列是这个术语的实现」和「这一列和这个术语相关」的区别。而这两件事在取数时的含义完全不同：前者可以直接用，后者只是线索。

这篇文章的留白。它在架构图里画了「反馈与记忆」这个组件，然后一句话跳过：「某些方面，比如反馈和记忆组件，在这次演练里不会实现」。而那恰好是语义层最难的一块——一个语义层用久了，会积累「这个问题上次用这几张表答对了」「那个术语其实对应的是另一列」这类经验，怎么把这些经验沉淀回图里而不让它变成一堆矛盾的标注，文章没有碰。

对我自己的启示。这篇和我手上 prove 那个项目是同一件事的两种做法，而对照之下的差别很值得记。

第一处对照，也是最要紧的一处：这个语义层是从数据库元数据自动生成的，而 prove 的语义层是人写的声明。

Neocarta 的路径是：连上 BigQuery，把库、模式、表、列的层级抽出来变成节点，把外键和 JOIN 日志变成边，再叠一层业务术语。**整个过程不需要人写一行本体声明。**而 prove 的做法是人写 tbox（概念）、rbox（关系）、指标注册表、映射注册表，加载期做 fail-fast 校验。

这两条路的代价完全不同，而且我此前没有把它想清楚过。自动生成的那条路覆盖面大、启动快，但它只能反映「数据库里有什么」，反映不了「业务上这个数该怎么算」——一列叫 complete 取值是 2 代表什么，外键和 JOIN 日志里都没有这个信息。而人写声明的那条路能表达口径，代价是每加一个指标都要有人写，而且写错了要靠校验去挡。

所以真正的问题不是「选哪条」，是「哪一部分该自动生成、哪一部分必须人写」。表结构层级、外键关系、列的物理类型——这些自动生成没有损失。而口径、同义词、业务含义、以及「这个数字该用哪个分母」——这些自动生成不出来。prove 现在是全部人写，其中有一部分本来可以省掉。

第二处对照：它那个「混合检索加业务术语」的两跳路径，和 prove 昨天刚做的「概念的表面说法」是同一个机制。

昨天在 prove 里做的事是：用户说「测评小节」而系统里那个概念叫 Exam，于是给概念加一层「用户会怎么称呼它」的声明。Neocarta 这里的 BusinessTerm 加 TAGGED_WITH 做的是同一件事——从用户的说法出发，两跳走到数据资产。

差别在于两点。**它的术语表来自 GCP Knowledge Catalog，是一份独立维护的资产**，而 prove 的表面说法写在本体里。它用向量检索加全文检索混合，而 prove 用的是精确匹配加同义词表。它的做法覆盖面更大但

会有误召回，prove 的做法精确但要人穷举说法。

第三处，一个直接的提醒：它把「反馈与记忆」画在架构图里然后跳过了。而 prove 现在也没有这一块——评测题里那些「这道题上次选错了指标」的记录，是靠 `known_fail` 标记和 KI 编号人工维护的，没有回流到本体里。这不是今天要解决的事，但值得知道：这一块在业界的公开方案里也是空着的，不是我们落后了。

文中金句

"The key attribute of the modern semantic layer is the relationships among the many disparate data assets." 现代语义层的关键属性，是众多互不相同的数据资产之间的关系。

"This augments the schema metadata graph with relationships that are not explicitly defined by foreign keys but can be instead inferred from JOIN conditions in SQL queries." 这一步给表结构元数据图补上了那些关系——它们没有被外键显式定义，但可以从 SQL 查询的 JOIN 条件里推断出来。

"This allowed the agent to answer increasingly complex questions about our data while keeping the context light." 这让智能体能够回答关于我们数据的越来越复杂的问题，同时保持上下文很轻。

全文翻译

一、引言：语义层为什么对智能体系统有用

Semantic layers are increasingly becoming an invaluable asset to agentic systems. They help an AI better understand the underlying data and enable it to handle large enterprise data landscapes. The key attribute of the modern semantic layer is the relationships among the many disparate data assets.

语义层正在越来越成为智能体系统的一项宝贵资产。它们帮助 AI 更好地理解底层数据，让 AI 能够处理大型企业的数据版图。现代语义层的关键属性，是众多互不相同的数据资产之间的关系。

What term does a column name resolve to? Are there other columns that share this meaning, and how can they be used to unlock novel insights? An agent with access to a graph-backed semantic layer has no problem connecting the dots and understanding where to find the answers in an ocean of data.

一个列名对应到哪个术语？还有别的列共享这个含义吗，它们怎么组合起来能得出新的洞察？一个能访问「图支撑的语义层」的智能体，在把点连起来、以及在一片数据海洋里搞清楚该去哪儿找答案这两件事上，都不成问题。

二、语义层架构

The semantic layer is first populated with source data in step 0. This includes a variety of data sources such as databases, data catalogs, query logs, supporting documents, and ontologies. This data provides the structure of the underlying data systems and facilitates understanding how to utilize it.

第 0 步先用源数据把语义层填起来。这包括各种数据源：数据库、数据目录、查询日志、支撑文档、以及本体。这些数据提供了底层数据系统的结构，并且方便理解怎么使用它。

Once data is loaded, the user may interact with a resource in the consumption layer, such as an agentic chat interface. This will kick off an agentic workflow that first retrieves context from the Neo4j semantic layer via the context MCP server. The agents in the agent layer will then use this context to select the appropriate query MCP servers and execute queries against the source databases. The agent layer then summarizes the results and returns a response to the user.

数据装载之后，用户可以和消费层的某个资源交互，比如一个智能体聊天界面。这会启动一条智能体工作流：先经由上下文 MCP 服务从 Neo4j 语义层取上下文；然后智能体层用这份上下文去选择合适的查询 MCP 服务，对源数据库执行查询；智能体层再把结果汇总，返回给用户。

Some aspects, such as the feedback and memory component, won't be implemented in this walkthrough.

某些方面，比如反馈与记忆组件，在这次演练里不会实现。

三、图数据模型

Before getting into the code, we should understand the shape of our semantic layer. There are two primary components we will implement.

在进入代码之前，我们应该先理解我们这个语义层的形状。我们要实现两个主要部分。

The foundation of our semantic layer is the metadata of our databases. Here, we explicitly map the database, schema, table, and column hierarchies in the graph. We also capture which columns relate to each other, either through foreign key definitions or by analyzing query patterns in the logs.

我们语义层的基础是数据库的元数据。在这里，我们把库、模式、表、列的层级关系显式地画进图里。我们还捕捉哪些列彼此相关——**要么通过外键定义，要么通过分析日志里的查询模式。**

Note: Database maps to the GCP project, and Schema maps to a BigQuery dataset.

注意：Database 对应 GCP 项目，Schema 对应一个 BigQuery 数据集。

The second component is glossary information, which provides business terminology. These entities allow for more advanced search patterns when it comes to context retrieval.

第二个部分是术语表信息，它提供业务术语。这些实体让上下文检索时可以用更高级的检索模式。

The glossary subgraph is layered over the metadata subgraph like so. BusinessTerm nodes are connected to Table and Column nodes via TAGGED_WITH relationships. This allows data assets to be identified via their connection to business terminology.

术语表子图就这样叠在元数据子图之上。BusinessTerm 节点通过 TAGGED_WITH 关系连接到 Table 和 Column 节点。这让数据资产可以经由它们与业务术语的连接被识别出来。

四、数据来源

Our semantic layer will have two primary data sources: BigQuery and Knowledge Catalog. These will provide the schema metadata and glossary graph components, respectively.

我们的语义层有两个主要数据源：BigQuery 和 Knowledge Catalog。它们分别提供表结构元数据和术语表这两个图的部分。

We will also demonstrate how to import metadata from query logs. This augments the schema metadata graph with relationships that are not explicitly defined by foreign keys but can be instead inferred from JOIN conditions in SQL queries.

我们还会演示怎么从查询日志导入元数据。[这一步给表结构元数据图补上那些关系——它们没有被外键显式定义，但可以从 SQL 查询的 JOIN 条件里推断出来。](#)

Note: The query log connector will also store Query and CTE nodes in our semantic layer. These may be referenced as few-shot examples.

注意：查询日志连接器还会在语义层里存下 Query 和 CTE 节点。这些可以作为少样本示例被引用。

Neocarta includes connectors for BigQuery, Dataplex, and query logs, making it easy to generate this graph. It also supports other sources such as OSI YAML files.

Neocarta 内置了 BigQuery、Dataplex 和查询日志的连接器，让生成这张图很容易。它还支持别的来源，比如 OSI YAML 文件。

五、建图（操作步骤概要）

演示用的数据集是 Neocarta 仓库里的 ACME Corp 数据集，含 33 张表、327 个列、76 个业务术语。整个建图过程是两条命令行：

```
uv run neocarta bigquery schema --embeddings
uv run neocarta dataplex glossary --embeddings
```

By running the Knowledge Catalog (Dataplex) connector second, we ensure that business terms will be connected to the existing schema components via TAGGED_WITH relationships.

把 Knowledge Catalog (Dataplex) 连接器放在第二个跑，可以确保业务术语会通过 TAGGED_WITH 关系连接到已经存在的表结构组件上。（换句话说，顺序不能反——术语要连的那些表和列必须先存在图里。）

可选的第三步是解析查询日志：

```
uv run neocarta bigquery logs --start-date 2026-01-01
```

最终图的形状是：左半边是 Database（深紫）到 Schema（粉）到 Table（黄绿）到 Column（橙）的层级，右半边是 Glossary（深紫）到 Category（棕）到 BusinessTerm（蓝）。

六、把智能体接到语义层

Since we have vector indexes, full-text indexes, and BusinessTerm nodes in our graph, the MCP server will register the following toolset: List schemas / List tables by schema / Get context by table business term hybrid search / Get context by column business term hybrid search / Get full metadata schema.

因为我们图里有向量索引、全文索引和 BusinessTerm 节点，MCP 服务会注册这样一套工具：列出模式；按模式列出表；按「表加业务术语」混合检索取上下文；按「列加业务术语」混合检索取上下文；取完整元数据模式（只用于开发和调试）。

The two hybrid search plus business term tools feature the most advanced context retrieval patterns. They will perform three searches to first identify the anchor node pool: Primary entity vector search / BusinessTerm full-text search / Primary entity full-text search.

那两个「混合检索加业务术语」的工具用的是最高级的上下文检索模式。它们会做三次检索来先确定锚点节点池：主实体的向量检索；BusinessTerm 的全文检索；主实体的全文检索。

The full-text search portion first identifies BusinessTerm nodes, then traverses to connected entity nodes (either Table or Column). Full-text search is then run again on these entity nodes, and the normalized average score is calculated. These results are then combined with the vector search results, ranked by score, and filtered to the top k.

全文检索这一路先找出 BusinessTerm 节点，然后顺着连接走到相连的实体节点（表或者列）。接着在这些实体节点上再跑一次全文检索，并计算归一化平均分。这些结果再与向量检索的结果合并、按分排序、筛到前 k 个。

Note: Full-text indexes are automatically generated for name and description properties, but can be turned off in the Python library connector arguments.

注意：全文索引会自动为 name 和 description 属性生成，但可以在 Python 库的连接器参数里关掉。

七、三个问答示例

第一个（两张表）： Which product line contributes the most to total ARR, and what is the breakdown?

哪条产品线对总年度经常性收入贡献最大，具体拆分是怎样的？

智能体的结论原文：

"Graph DB Enterprise is the dominant revenue driver, accounting for over 80% of ARR despite having fewer subscriptions than Cloud. This concentration in a single product line is a key business risk to monitor."

Graph DB Enterprise 是收入的主导来源，占年度经常性收入八成以上，尽管它的订阅数比 Cloud 少。这种集中在单一产品线上的情况是一项需要监控的关键业务风险。

第二个（三张表）： How are performance ratings distributed across departments in the most recent review cycle?

最近一个考核周期里，绩效评级在各部门之间是怎么分布的？

"Engineering has the widest performance spread — three 'Exceeds' employees averaging 4.6, alongside one 'Below' employee at 2.4. Sales and Product skew positive. Customer Success has only one reviewed employee, suggesting a data coverage gap to investigate."

工程部门的绩效跨度最大——三名「超出预期」的员工平均 4.6 分，同时有一名「低于预期」的员工 2.4 分。销售和产品偏正面。客户成功部门只有一名被考核的员工，[这提示存在一个需要调查的数据覆盖缺口](#)。

第三个（需要推断）： Is there a correlation between a customer's health score and their support experience?

客户的健康分和他们的支持体验之间有相关性吗？

"A clear positive correlation emerges between health score and support quality. The churned customer (Umbrella Biotech, score 30) had the lowest CSAT (3.0) and the longest resolution time (49 hrs) of any rated ticket. Conversely, the two highest-scoring customers had perfect CSAT scores and the fastest resolution times. This suggests that support

responsiveness is both a leading indicator of and contributor to customer health. Customers with CSAT scores below 4.0 or resolution times above 30 hours warrant proactive CSM outreach."

健康分与支持质量之间出现了明显的正相关。那个已流失的客户（Umbrella Biotech，健康分 30）在所有被评分的工单里客服满意度最低（3.0）、解决时长最长（49 小时）。反过来，两个健康分最高的客户拿到了满分的客服满意度和最快的解决时长。这说明[支持响应速度既是客户健康度的领先指标，也是它的成因之一](#)。客服满意度低于 4.0 或解决时长超过 30 小时的客户，值得客户成功经理主动接触。

八、小结

In this walkthrough, we demonstrated how to build a semantic layer in Neo4j using the Neocarta Python library. We integrated BigQuery schema metadata and Knowledge Catalog glossary information to create a rich, traversable map of our data landscape. This allowed the agent to answer increasingly complex questions about our data while keeping the context light.

在这次演练里，我们演示了怎么用 Neocarta 这个 Python 库在 Neo4j 里建一个语义层。我们把 BigQuery 的表结构元数据和 Knowledge Catalog 的术语表信息整合起来，做出一张内容丰富、可遍历的数据版图。[这让智能体能够回答关于我们数据的越来越复杂的问题，同时保持上下文很轻。](#)

参考链接

- [neo4j-labs/neocarta](#) — 本文用的库，Neo4j Labs 项目，本文锁在 v0.7.0；仍在快速开发中